

AIOps Module 1 Assignment

Name: Krish Jeevankumar Yadav **Roll Number:** DA24B043

Question 2 - Applied: MLflow Experiment Comparison

1. MLflow Run Comparison

(See the attached `mlflow_comparison.png` in the repository for the table of all six runs).

2. Run Analysis

The MLflow run-comparison table shows the best-performing experiment was the run configured with a learning rate of 0.001 and a batch size of 32 (`mlp-lr0.001-bs32`). It achieved the highest validation accuracy at 94.1% (`val_accuracy`: 0.941).

There is clear evidence of overfitting when comparing the final training loss against the validation accuracy trend. In the best-performing run, the `train_loss` plummeted to an extreme low of 0.002, which indicates the MLP practically memorized the training data. Despite this near-perfect training score, the validation accuracy capped out at 94.1%. Similarly, the run with a learning rate of 0.001 and batch size of 128 achieved a near-zero training loss of 0.013, but its validation accuracy stalled at 93.7%. This massive divergence between training loss approaching zero while validation accuracy plateaus is a definitive signal that the model stopped generalizing effectively to unseen data.

Between the two varied hyperparameters, the learning rate had a significantly larger impact on overall performance than the batch size. Shifting the learning rate from 0.0001 to 0.001 triggered a $\sim 4\%$ jump in accuracy. By contrast, toggling the batch size between 32 and 128 within the same learning rate bracket only nudged the final accuracy by roughly 1% to 2%.

3. Logging Code Added

```
mlflow.log_param("learning_rate_init", lr)
mlflow.log_param("batch_size", bs)
mlflow.log_param("model_type", "MLPClassifier")

mlflow.log_metric("train_loss", train_loss, step=epoch)
mlflow.log_metric("val_accuracy", val_accuracy, step=epoch)

mlflow.log_metric("accuracy", accuracy_score(y_test, final_preds)
)
mlflow.log_metric("f1_macro", f1_score(y_test, final_preds,
    average="macro"))
```